Assignment: Assignment 4 - Facade Design Pattern Implementation

Agabekov Akzhigit SE2523

1. Introduction

The primary objective of this assignment is to design and implement a Java application demonstrating the **Facade Structural Design Pattern**.

Why the Subsystem Needs a Facade

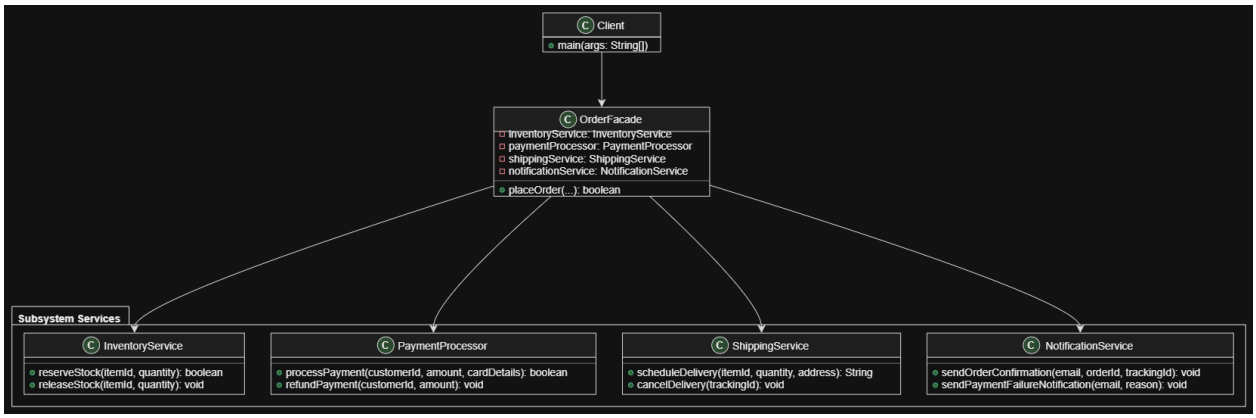
In an e-commerce platform, placing a customer order requires coordinating four separate subsystem services:

1. **InventoryService**: Manages product stock levels, stock reservations, and stock releases.
2. **PaymentProcessor**: Handles credit/debit card transactions and processes refunds.
3. **ShippingService**: Schedules parcel shipments with carrier logistics and generates tracking IDs.
4. **NotificationService**: Dispatches confirmation emails or payment failure alerts to customers.

Without a Facade, the Client (Main) would need to directly instantiate, configure, and orchestrate all four services in a specific sequence. Furthermore, the client would be forced to manage error conditions and transactional rollbacks manually (e.g., releasing reserved inventory if the credit card payment is declined).

The OrderFacade class solves this complexity by providing a unified, simplified interface (placeOrder()) that encapsulates all orchestration and rollback logic.

2. Structural Architecture & UML Diagrams



3. Clean Code Principles Applied

1 Meaningful Naming (Clean Code Ch. 2):

All method identifiers (placeOrder(), reserveStock(), processPayment()) explicitly express business intent without ambiguous abbreviations or misleading terms.

2 Single Responsibility Principle (SRP - Clean Code Ch. 3):

The OrderFacade is strictly responsible for workflow coordination. It delegates domain-specific logic (e.g., payment validation, stock mutation) to individual subsystem services, keeping methods small and focused.

3 Encapsulation & No Detail Leakage:

The public signature of OrderFacade.placeOrder() consumes domain primitives (String, int, double) and hides subsystem object creation, execution order, and failure states from the client

4 Small, Focused Subsystem Classes (Clean Code Ch. 6):

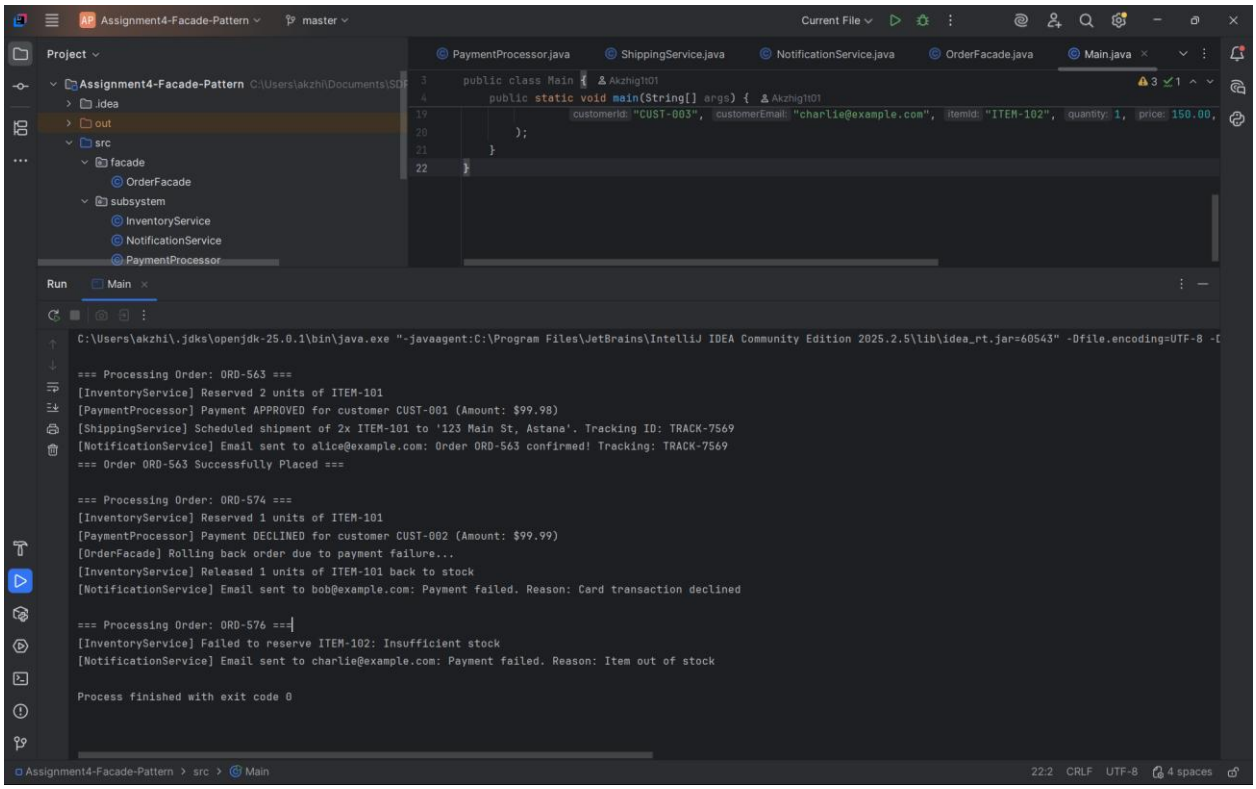
Each service class (InventoryService, ShippingService, etc.) is lightweight and manages only its domain responsibilities without back-references to the Facade

5 Proper Error Handling & Transactional Rollback (Clean Code Ch. 7):

If PaymentProcessor.processPayment() fails, OrderFacade catches the error condition and triggers a rollback step (inventoryService.releaseStock()), preserving data consistency

4. Verification & Output Log

Executing Main.java produced the following terminal output trace:



5. Conclusion & Reflections

What the Facade Hides: The OrderFacade completely hides subsystem interactions, service initialization, execution ordering, and rollback error handling.

What Remains Directly Accessible: Subsystem classes remain independent public classes without back-references to OrderFacade. Advanced clients who require custom workflows can still instantiate and interact with InventoryService or PaymentProcessor directly.